TECNO 3F | Programá tu futuro

Municipalidad de Tres de Febrero

www.tresdefebrero.gov.ar/**tecno3f**

#ProgramáTuFuturo

TECNO3F | Programá tu futuro

3F | Municipalidad de Tres de Febrero

Municipalidad de Tres de Febrero

TECNO 3F | Programá tu futuro

PYTHON INTERMEDIO

¡Les damos la bienvenida!

Municipalidad de Tres de Febrero

TECNO 3F | Programá tu futuro

PROGRAMACIÓN ORIENTADA A OBJETOS II

CLASE 5

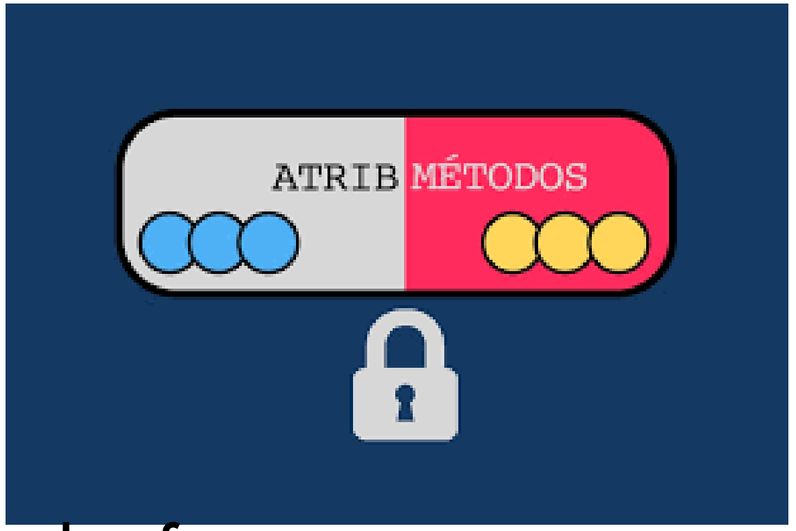
ENCAPSULAMIENTO

TECNO3F | Programá tu futuro

3F | Municipalidad de Tres de Febrero

¿QUÉ ES?

Consiste en agrupar los datos (atributos) y los comportamientos (métodos) en una misma entidad, controlando y restringiendo el acceso directo a las variables internas desde el exterior.



El problema sin encapsular:
Si dejamos los atributos abiertos, cualquiera podría hacer desde afuera:
mi_auto.velocidad = -200 o mi_auto.combustible = 150
dejando al vehículo en un estado ilógico o roto.

La solución encapsulada:
Protegemos las variables internas (_velocidad, _combustible).
Obligamos a interactuar mediante métodos públicos (acelerar(), frenar(), cargar_combustible()) que aplican reglas y validaciones de seguridad.

Como se hace en python

Convención de nomenclatura (PEP 8):
Python no utiliza palabras clave como private o protected. El acceso se define mediante convenciones:

- Público (marca): Accesible desde cualquier parte del programa.
- Protegido (_velocidad): Un guión bajo indica que es de uso interno de la clase y de sus clases hijas. Señal de: "no lo toques directo desde afuera".
- Privado con Mangling (__numero_motor): Doble guión bajo. Python renombra la variable internamente para evitar sobrescrituras accidentales.

```python
class Vehiculo:
    def __init__(self, marca, modelo):
        #Atributo público (lectura/escritura libre)
        self.marca = marca
        self.modelo = modelo
        #Atributo protegido (convención interna)
        self._velocidad = 0

        #Atributo privado estricto (Name Mangling)
        self.__numero_chasis = "CH-98432"

auto = Vehiculo("Toyota", "Corolla")
print(auto.marca)       #"Toyota"
print(auto._velocidad)  # Funciona, pero viola la convención
# print(auto.__numero_chasis) #Lanza AttributeError
```

El Enfoque Tradicional: Getters y Setters

En otros lenguajes (como Java o C#), para proteger un atributo se lo privatiza y se crean dos métodos por cada variable:

- Un Getter para leer el valor.
- Un Setter para modificarlo aplicando validaciones.

```python
# Getter clásico
def get_velocidad_maxima(self):
    return self._velocidad_maxima

# Setter clásico con validación
def set_velocidad_maxima(self, valor):
    if valor <= 0:
        print("La velocidad máxima debe ser positiva.")
        return
    self._velocidad_maxima = valor
```

```python
auto.set_velocidad_maxima(200)
print(auto.get_velocidad_maxima())
```

Encapsulamiento Pythónico con @property

El decorador @property permite que un método se comporte y se lea externamente como si fuera un atributo directo, pero ejecutando internamente validaciones de seguridad:

- @property: Funciona como el Getter (lectura).
- @nombre_atributo.setter: Funciona como el Setter (escritura con control).

```python
#Getter decorado
@property
def velocidad_maxima(self):
    return self._velocidad_maxima

#Setter decorado
@velocidad_maxima.setter
def velocidad_maxima(self, valor):
    if valor <= 0:
        raise ValueError("La velocidad máxima debe ser positiva.")
    self._velocidad_maxima = valor
```

```
auto.velocidad_maxima = 220
auto.velocidad_maxima = -50
```

HERENCIA

TECNO 3F | Programá tu futuro

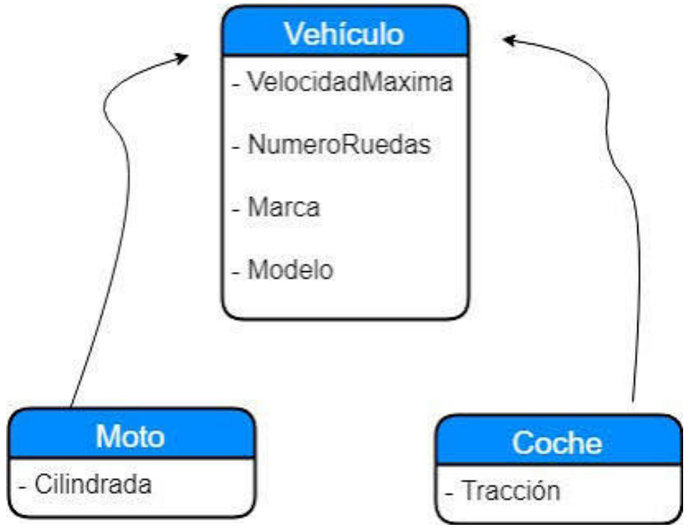
3F | Municipalidad de Tres de Febrero

¿QUÉ ES?

Mecanismo que permite crear una nueva clase (subclase o clase hija) reutilizando todos los atributos y métodos de una clase preexistente (superclase o clase padre)

Usamos herencia únicamente cuando la subclase es un tipo específico de la clase padre:

- Un Auto es un Vehiculo.
- Una Moto es un Vehiculo.
- Un Camion es un Vehiculo.



No reescribimos lógica común (marca, modelo, encender(), frenar()). Todo vive en la clase padre y las hijas solo agregan lo que las hace únicas.

¿Cómo heredamos en Python?

Mecanismo que permite crear una nueva clase (subclase o clase hija) reutilizando todos los atributos y métodos de una clase preexistente (superclase o clase padre)

```python
#Creamos la clase Hija e indicamos de que clase Hereda
class Moto(Vehiculo):
    pass
```

La clase hija adquiere de forma inmediata todos los atributos y métodos de la clase padre.

```python
mi_moto = Moto("Yamaha", "MT-03")

print(mi_moto.velocidad_maxima)
```

Extendiendo la clase hija con super()

Casi siempre la subclase necesita atributos propios (por ejemplo, un auto tiene cantidad_puertas y una moto tiene cilindrada)

Si creamos un nuevo **__init__** en la hija, pisamos el del padre.

Para evitar duplicar código, usamos la función super().__init__(...): le delega a la clase padre la inicialización de sus datos base y luego definimos los datos exclusivos de la hija.

```python
#Creamos la clase Hija e indicamos de que clase Hereda
class Auto(Vehiculo):
    def __init__(self, marca, modelo, velocidad_maxima = 75,cantidad_puertas=4):
        #Delegamos al constructor de Vehiculo
        super().__init__(marca, modelo,velocidad_maxima)

        #Agregamos el atributo exclusivo de Auto
        self.cantidad_puertas = cantidad_puertas

    #Agregamos un método exclusivo de Auto
    def tocar_bocina(self):
        print(f"El {self.marca} hace: ¡Pip, pip!")


mi_auto = Auto("Peugeot", "208",cantidad_puertas=5)

print(mi_auto.velocidad_maxima)
print(f"Puertas: {mi_auto.cantidad_puertas}")
mi_auto.tocar_bocina()
```

POLIMORFISMO

TECNO3F | Programá tu futuro

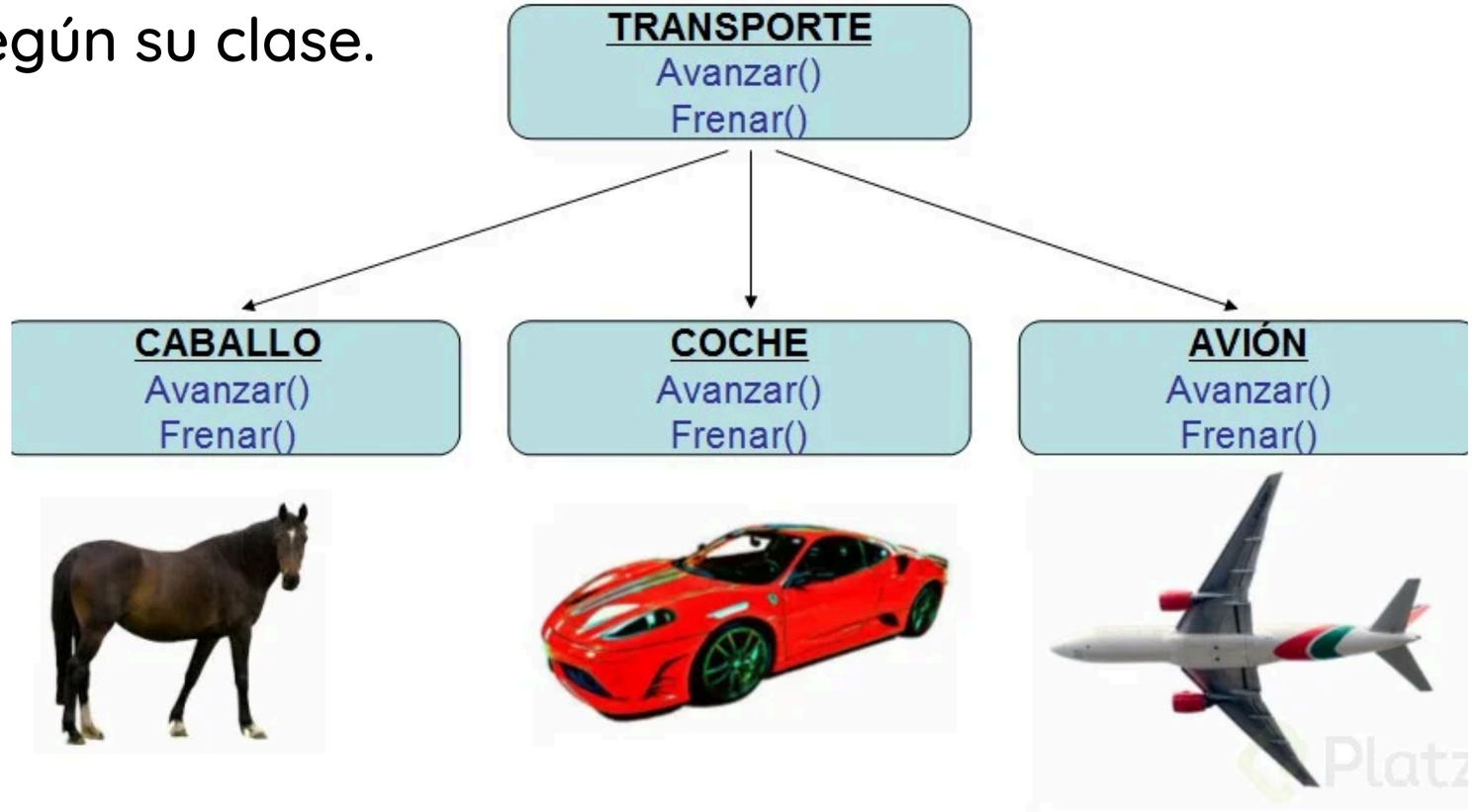
3F | Municipalidad de Tres de Febrero

¿QUÉ ES?

Poli (muchas) + morfo (formas).

Es la capacidad que tienen diferentes objetos de responder al mismo método con comportamientos específicos según su clase.

Podemos escribir código genérico (como un bucle o una función) que opere sobre una lista de vehículos sin importarle qué tipo exacto de vehículo es cada uno, delegando en cada objeto la ejecución de su propia versión del método.



¿Por qué usamos Polimorfismo?

El enfoque rígido

```
for v in flota:
    if isinstance(v, Auto):
        print("Acelerando auto de forma estándar...")
    elif isinstance(v, Camion):
        print("Acelerando camión con peso...")
    elif isinstance(v, Moto):
        print("Acelerando moto rápido...")
```

El enfoque polimórfico

```
for v in flota:
    v.acelerar(30)
```

Ventaja: Si mañana sumás Colectivo o Avion, no tocás el bucle ni agregás ningún elif. El sistema es extensible y desacoplado.

CLASE ABSTRACTA

TECNO 3F | Programá tu futuro

3F | Municipalidad de Tres de Febrero

¿Cómo se define en Python?

En Python las clases abstractas no son una palabra reservada del lenguaje (como en Java/C#), sino que se implementan mediante el módulo estándar abc (Abstract Base Classes).

Importamos dos elementos esenciales:

- ABC: Clase base de la cual debe heredar nuestra clase para volverse abstracta.
- @abstractmethod: Decorador que colocamos sobre los métodos que obligatoriamente deben implementar las clases hijas.

```
from abc import ABC, abstractmethod
```

Creando la Clase Abstracta Vehiculo

- acemos que Vehiculo herede de ABC.
- Conserva sus atributos comunes (marca, modelo, _velocidad).
- Definimos métodos concretos (como describir()) y marcamos con @abstractmethod aquellos que no tienen lógica general y obligan a cada vehículo a definir su propia versión.

```python
from abc import ABC, abstractmethod

class Vehiculo(ABC):
    def __init__(self, marca, modelo,velocidad_maxima = 50):
        self.marca = marca
        self.modelo = modelo
        self._velocidad = 0
        self._velocidad_maxima = velocidad_maxima
        self.__numero_chasis = "CH-98432"

    @property
    def velocidad_maxima(self):
        return self._velocidad_maxima

    @velocidad_maxima.setter
    def velocidad_maxima(self, valor):
        if valor <= 0:
            raise ValueError("La velocidad máxima debe ser positiva.")
        self._velocidad_maxima = valor

    def acelerar(self, incremento):
        self._velocidad = min(self._velocidad + incremento, self._velocidad_maxima)
        print(f"{self.marca} {self.modelo} acelera a {self._velocidad} km/h.")

    def __str__(self):
        return f"{self.marca} {self.modelo}"

    @abstractmethod
    def inspeccion_tecnica(self):
        """Cada subclase DEBE implementar sus propios controles."""
        pass
```

¿Qué pasa si intentamos instanciarla?

En el mundo real, nadie compra "un vehículo genérico"; compramos un auto, una moto o un camión. Vehiculo es solo un concepto abstracto.

```python
vehiculo_generico = Vehiculo("Generico", "X1")

print(vehiculo_generico.describir())
```

```
Traceback (most recent call last):
  File "c:\Users\magog\Documents\Tecno3F\python\inter\2026\6\vehiculo2.py", line 64, in <module>
    vehiculo_generico = Vehiculo("Generico", "X1")
TypeError: Can't instantiate abstract class Vehiculo without an implementation for abstract method 'inspeccion_tecnica'
```

No podemos instanciar una clase abstracta.

Si una clase tiene al menos un @abstractmethod, Python bloquea su creación directa. Su único propósito es servir como molde base para que otras clases la hereden e implementen los métodos faltantes.

METODO MAGICO

TECNO 3F | Programá tu futuro

3F | Municipalidad de Tres de Febrero

El Método Especial __str__

Cuando hacemos print(objeto), Python intenta mostrarlo en pantalla. Si no definimos __str__, muestra una dirección de memoria cruda e incomprensible

Es un método mágico (dunder) que define cómo queremos que se vea el objeto convertido a texto (para humanos).

Siempre debe retornar un str (texto con return, nunca hacer print dentro de él)

¡MUCHAS GRACIAS!

TECNO3F | Programá tu futuro

3F | Municipalidad de Tres de Febrero